

**ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
ІМЕНІ ІВАНА ФРАНКА**

**ЗВІТ
ДО ЛАБОРАТОРНОЇ РОБОТИ №1
«Організація професійного Python-проєкту та Git»
з курсу «Професійний Python»
Варіант №4 — «Система банківських рахунків»**

Виконав(ла): Фрадкова А.С.

Група: ФЕП-32

Перевірів(ла): доц. Куньо І.М.

Дата: 15.09.2026

1. Мета роботи

Ознайомитися з основними принципами організації професійного Python-проєкту, навчитися створювати ізольоване середовище розробки, формувати структуру проєкту за принципом src-layout, працювати з модулями та пакетами Python, створювати файл pyproject.toml, використовувати Git для контролю версій та оформлювати базову документацію проєкту.

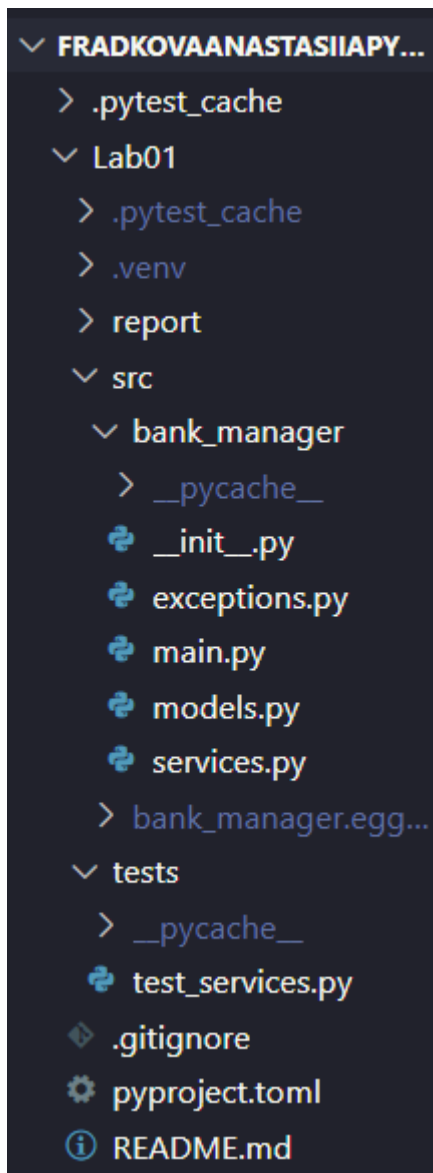
2. Індивідуальний варіант і постановка задачі

Варіант №4 — «Система банківських рахунків». Для кожного рахунку зберігати ім'я клієнта, номер рахунку, баланс. Реалізувати поповнення рахунку, списання коштів, перевірку балансу, пошук рахунку, обчислення загальної суми коштів. Не дозволяється отримувати від'ємний баланс.

Проєкт повинен мати src-layout, щонайменше три Python modules, data model, окремий module з business logic, main.py, type hints, README.md, .gitignore, pyproject.toml і Git repository.

3. Структура проєкту

```
Lab01/
├── pyproject.toml
├── README.md
├── .gitignore
├── src/
│   ├── bank_manager/
│   │   ├── __init__.py
│   │   └── main.py
│   ├── exceptions.py
│   ├── models.py
│   └── services.py
└── tests/
    └── test_services.py
```



Каталог `src/bank_manager` містить пакет застосунку, `models.py` описує модель даних `Student`, `services.py` містить business logic, `main.py` є точкою входу, а `tests/test_services.py` містить автоматизовані тести.

4. Опис створених модулів

4.1. models.py

Для представлення банківського рахунку створено датаклас `Account` з полями `number`, `client_name` і `balance`.

```
from dataclasses import dataclass
```

```

@dataclass(frozen=True, slots=True)
class Account:
    number: int
    client_name: str
    balance: float

```

4.2. services.py

У модулі реалізовано функції `top_up()`, `withdraw()`, `check_balance()`, `find_account()`, `calculate_total_sum()`. Бізнес-логіка не взаємодіє з консоллю.

```

from bank_manager.models import Account
from bank_manager.exceptions import InsufficientFundsError
from dataclasses import replace

def top_up(account: Account, sum: float) -> Account:
    return replace(account, balance=account.balance + sum)

def withdraw(account: Account, sum: float) -> Account | None:
    if account.balance < sum:
        raise InsufficientFundsError(f"Balance too small to withdraw
{sum}")
    return replace(account, balance=account.balance - sum)

def check_balance(account: Account) -> float:
    return account.balance

def find_account(accounts: list[Account], number: int) -> Account |
None:
    for account in accounts:
        if account.number == number:
            return account
    return None

```

```
def calculate_total_sum(accounts: list[Account]) -> float:
    return sum(account.balance for account in accounts)
```

4.3. main.py

Модуль main.py створює демонстраційні дані, викликає функції з services.py та виводить результати. Конструкція if __name__ == "__main__": відокремлює точку входу від функцій програми.

```
from bank_manager.models import Account

from bank_manager.services import top_up, withdraw, check_balance,
find_account, calculate_total_sum

from bank_manager.exceptions import InsufficientFundsError


def create_accounts() -> list[Account]:
    return [
        Account(number=1, client_name='Alice', balance=25.5),
        Account(number=2, client_name='Bob', balance=10.0)
    ]


def print_account_info(account: Account) -> None:
    print(f"Name: {account.client_name}, number: {account.number},
balance: {account.balance}")


def main() -> None:
    accounts = create_accounts()

    print("INITIAL BALANCES")

    for account in accounts:
        print_account_info(account)

    print()

    print("TOP UP")
```

```
accounts[0] = top_up(accounts[0], 3.0)

print_account_info(accounts[0])

print()

print("FAILED WITHDRAWAL")

try:
    accounts[0] = withdraw(accounts[0], 100.0)
except InsufficientFundsError as e:
    print(e)

print()

print("SUCCESSFUL WITHDRAWAL")

try:
    accounts[0] = withdraw(accounts[0], 20.0)
    print_account_info(accounts[0])
except InsufficientFundsError as e:
    print(e)

print()

print("ACCOUNT SEARCH")

target_number = 2

account = find_account(accounts, target_number)

if account:
    print(f"Found account {target_number}.")
    print_account_info(account)
else:
    print(f"Account {target_number} not found.")

print()

print("TOTAL SUM CALCULATION")

print(f"Total sum: {calculate_total_sum(accounts):.2f}")
```

```
if __name__ == "__main__":  
    main()
```

4.4. test_services.py

Додано pytest-тести для перевірки функцій поповнення, списання коштів, пошуку рахунку та підрахунку балансу.

```
import pytest  
  
from bank_manager.models import Account  
  
from bank_manager.services import top_up, withdraw, check_balance,  
find_account, calculate_total_sum  
  
from bank_manager.exceptions import InsufficientFundsError  
  
@pytest.fixture  
  
def sample_accounts() -> list[Account]:  
  
    return [  
  
        Account(number=1, client_name="Alice", balance=100.0),  
  
        Account(number=2, client_name="Bob", balance=50.0)  
  
    ]  
  
def test_top_up():
```

```
acc = Account(number=1, client_name="Alice", balance=100.0)

updated_acc = top_up(acc, 50.0)

assert updated_acc.balance == 150.0

assert updated_acc.number == 1


def test_withdraw_success():

    acc = Account(number=1, client_name="Alice", balance=100.0)

    updated_acc = withdraw(acc, 40.0)

    assert updated_acc is not None

    assert updated_acc.balance == 60.0


def test_withdraw_insufficient_funds():

    acc = Account(number=1, client_name="Alice", balance=100.0)

    with pytest.raises(InsufficientFundsError):

        withdraw(acc, 200.0)
```

```
def test_check_balance():

    acc = Account(number=1, client_name="Alice", balance=123.45)

    assert check_balance(acc) == 123.45


def test_find_account_success(sample_accounts):

    found = find_account(sample_accounts, 2)

    assert found is not None

    assert found.client_name == "Bob"


def test_find_account_not_found(sample_accounts):

    not_found = find_account(sample_accounts, 99)

    assert not_found is None


def test_calculate_total_sum(sample_accounts):

    total = calculate_total_sum(sample_accounts)

    assert total == 150.0
```

```
MINGW64:/c:/Anastasiia/Development/University/Професійний пайтон/Fradk...
configfile: pyproject.toml
collecting ... collected 7 items

tests/test_services.py::test_top_up PASSED [ 14%]
tests/test_services.py::test_withdraw_success PASSED [ 28%]
tests/test_services.py::test_withdraw_insufficient_funds PASSED [ 42%]
tests/test_services.py::test_check_balance PASSED [ 57%]
tests/test_services.py::test_find_account_success PASSED [ 71%]
tests/test_services.py::test_find_account_not_found PASSED [ 85%]
tests/test_services.py::test_calculate_total_sum PASSED [100%]

===== 7 passed in 0.02s =====
(.venv)
```

5. Файл pyproject.toml

Файл задає build system setuptools, основні метадані проєкту (назву, версію та опис), вимогу до версії Python 3.11+ та console script bank-manager для запуску програми.

```
Lab01 > ⚙ pyproject.toml
1  [build-system]
2  requires = ["setuptools>=70"]
3  build-backend = "setuptools.build_meta"
4
5  [project]
6  name = "bank-manager"
7  version = "0.1.0"
8  description = "Bank accounts management laboratory project"
9  requires-python = ">=3.11"
10
11 [project.scripts]
12 bank-manager = "bank_manager.main:main"
```

6. Файл .gitignore

З Git виключено virtual environment, кеші, IDE configuration, build artifacts і .env.

```
Lab01 > .gitignore
1  .venv/
2  __pycache__/
3  *.pyc
4  .pytest_cache/
5  .idea/
6  .vscode/
7  dist/
8  build/
9  *.egg-info/
```

7. README.md

```
# Bank Manager (Lab 01)

## Description
Console application for managing bank accounts and financial operations.

## Requirements
Python 3.11+

## Installation
python -m venv .venv
.venv\Scripts\Activate.ps1
python -m pip install -e .

## Run
python -m bank_manager.main

## Author
Full name: Anastasiia Fradkova
Group: FEP-32
```

8. Створення virtual environment та запуск

```
MINGW64:/c/Anastasiia/Development/University/Професійний пайтон/FradkovaAnastasiiaPython/Lab01

Anastasiia@PHANTOM-OF-VENICE MINGW64 /c/Anastasiia/Development/University/Професійний пайтон/FradkovaAnastasiiaPython/Lab01 (master)
$ python -m venv .venv

Anastasiia@PHANTOM-OF-VENICE MINGW64 /c/Anastasiia/Development/University/Професійний пайтон/FradkovaAnastasiiaPython/Lab01 (master)
$ source .venv/Scripts/activate
(.venv)
```

```
PS C:\Anastasiia\Development\University\Професійний пайтон\FradkovaAnastasiiaPython\Lab01> python -m pip install -e .
Defaulting to user installation because normal site-packages is not writeable
Obtaining file:///C:/Anastasiia/Development/University/%D0%9F%D1%80%D0%BE%D1%84%D0%B5%D1%81%D1%96%D0%B9%D0%BD%D0%B8%D0%B9%20aPython/Lab01
Installing build dependencies ... done
Checking if build backend supports build_editable ... done
Getting requirements to build editable ... done
Preparing editable metadata (pyproject.toml) ... done
Building wheels for collected packages: bank-manager
Building editable for bank-manager (pyproject.toml) ... done
Created wheel for bank-manager: filename=bank_manager-0.1.0-0.editable-py3-none-any.whl size=1606 sha256=8315a8917d9eb54f7
```

9. Результат роботи програми

```
PS C:\Anastasiia\Development\University\Професійний пайтон\FradkovaAnastasiiaPython\Lab01> python -m bank_manager.main
INITIAL BALANCES
Name: Alice, number: 1, balance: 25.5
Name: Bob, number: 2, balance: 10.0

TOP UP
Name: Alice, number: 1, balance: 28.5

FAILED WITHDRAWAL
Balance too small to withdraw 100.0

SUCCESSFUL WITHDRAWAL
Name: Alice, number: 1, balance: 8.5

ACCOUNT SEARCH
Found account 2.
Name: Bob, number: 2, balance: 10.0

TOTAL SUM CALCULATION
Total sum: 18.50
```

10. Робота з Git

У проєкті було створено 6 комітів:

git init

git add .

git commit -m "Create Lab01 project structure"

```
git add .
```

```
git commit -m "Add account model and business logic services"
```

```
git add .
```

```
git commit -m "Add application entry point"
```

```
git add .
```

```
git commit -m "Add documentation"
```

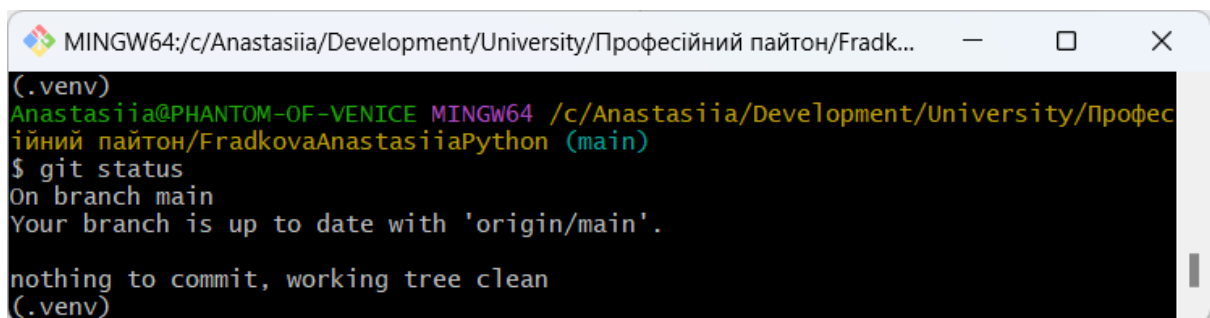
```
git add .
```

```
git commit -m "Add pytest tests and custom exceptions"
```

```
git add .
```

```
git commit -m "Modify pyproject.toml"
```

10.1. Результат git status

A screenshot of a terminal window with a dark background. The window title is "MINGW64:/c/Anastasiia/Development/University/Професійний пайтон/Fradk...". The terminal shows the following text:

```
(.venv)  
Anastasiia@PHANTOM-OF-VENICE MINGW64 /c/Anastasiia/Development/University/Професійний пайтон/FradkovaAnastasiiaPython (main)  
$ git status  
On branch main  
Your branch is up to date with 'origin/main'.  
  
nothing to commit, working tree clean  
(.venv)
```

10.2. Результат git log --oneline

```
MINGW64:/c/Anastasiia/Development/University/Професійний пайтон/FradkovaAnastasiiaPython (main)
$ git log --oneline
efb5798 (HEAD -> main, origin/main) Modify pyproject.toml
4aa6f3f Add pytest tests and custom exceptions
b82dc28 Add the report
a660080 Add documentation
7d5d029 Add application entry point
c0ed119 Add Account model and business logic services
88b47af Create Lab01 project structure
(.venv)
Anastasiia@PHANTOM-OF-VENICE MINGW64 /c/Anastasiia/Development/University/Професійний пайтон/FradkovaAnastasiiaPython (main)
$
```

11. Відповіді на контрольні запитання

1. Що таке Python-проект?

Це структурований набір вихідного коду, супровідної документації, конфігураційних файлів та тестів, що складають цілісний програмний продукт.

2. Чим проєкт відрізняється від одного Python script?

На відміну від звичайного скрипта, повноцінний проєкт розбитий на модулі, містить засоби тестування, власну документацію та налаштовані залежності.

3. Що таке Python module?

Звичайний файл із розширенням .py, у якому написаний код на Python.

4. Що таке Python package?

Директорія, яка слугує контейнером для логічно пов'язаних між собою Python-модулів.

5. Для чого потрібний __init__.py?

Він вказує інтерпретатору, що папку слід обробляти як пакет, а також може містити початкові налаштування чи API.

6. Що таке src-layout?

Архітектурний підхід, за якого весь розроблюваний код зберігається у спеціалізованій папці src.

7. Які переваги має src-layout?

Такий підхід робить структуру проєкту більш упорядкованою та запобігає помилкам під час імпорту та тестування пакета.

8. Для чого використовується virtual environment?

Щоб відокремити бібліотеки та їхні версії для конкретного проєкту від глобального середовища системи.

9. Як створити .venv?

За допомогою команди `python -m venv .venv` у терміналі.

10. Як активувати virtual environment?

На ОС Windows це робиться скриптом `.venv\Scripts\activate`, а на UNIX-системах – командою `source .venv/bin/activate`.

11. Для чого використовується pip?

Це пакетний менеджер, призначений для встановлення та оновлення сторонніх бібліотек Python.

12. Чому рекомендується python -m pip?

Цей синтаксис гарантує, що пакети будуть встановлені саме для того інтерпретатора, який зараз активний у віртуальному середовищі.

13. Для чого потрібний pyproject.toml?

Це сучасний файл стандарту для зберігання налаштувань інструментарію, залежностей та загальної інформації про застосунок.

14. Що містить секція [project]?

Базову інформацію (автор, назва, версія), а також перелік необхідних залежностей і мінімальну версію Python.

15. Що таке editable installation?

Це режим розробки (через прапорець `-e`), який дозволяє застосовувати зміни у вихідному коді без необхідності перевстановлювати пакет.

16. Для чого використовується if __name__ == "__main__"?

Ця перевірка гарантує, що основний код виконається лише тоді, коли файл запускають напямую, а не при його імпорті в інші модулі.

17. Що таке Git?

Популярна розподілена система, що дозволяє відстежувати зміни в коді та працювати над ним спільно.

18. Що таке repository?

Директорія проєкту, яка відстежується системою Git і містить усю історію розробки.

19. Що таке commit?

Це збережений стан проєкту (зліпок файлів) у певний момент часу з унікальним ідентифікатором.

20. Для чого використовується git status?

Ця команда показує, які файли були змінені, додані або підготовлені до фіксації.

21. Для чого git add?

Щоб додати зміни до staging area.

22. Для чого git commit?

Щоб остаточно зберегти проіндексовані файли в історію репозиторію.

23. Що показує git log?

Хронологічний список усіх зроблених комітів.

24. Для чого потрібен .gitignore?

У ньому прописують шляхи до файлів (кеш, паролі, середовища), які система контролю версій має ігнорувати.

25. Чому .venv не потрібно комітити?

Ця папка важить багато, містить файли для конкретної ОС, і її завжди можна легко згенерувати заново на іншому комп'ютері.

26. Для чого потрібний README?

Це головний текстовий файл із документацією, який пояснює, що робить застосунок і як ним користуватися.

27. Що таке type hints?

Це головний текстовий файл із документацією, який пояснює, що робить застосунок і як ним користуватися.

28. Для чого програму розділяють на modules?

Щоб код був більш структурованим, легко читався, простіше тестувався та масштабувався.

29. Що таке business logic?

Основний алгоритмічний код, який реалізує ключові задачі та правила конкретного застосунку.

30. Чому потрібно відокремлювати application entry point від business logic?

Розділення дозволяє легко покривати бізнес-логіку автотестами або ж змінювати інтерфейс без втручання в самі алгоритми.

12. Висновок

Ознайомилася з основними принципами організації професійного Python-проєкту, навчилася створювати ізольоване середовище розробки, формувати структуру проєкту за принципом src-layout, працювати з модулями та пакетами Python, створювати файл pyproject.toml, використовувати Git для контролю версій та оформлювати базову документацію проєкту.